Quantum Amplitude Loading for Rainbow Options Pricing

Francesca Cibrario*, Or Samimi Golan†, Giacomo Ranieri*, Emanuele Dri‡, Mattia Ippoliti*, Ron Cohen†, Christian Mattia*, Bartolomeo Montrucchio‡, Amir Naveh†, and Davide Corbelletto*

*Intesa Sanpaolo, Torino, Italy
francesca.cibrario@intesasanpaolo.com, giacomo.ranieri@intesasanpaolo.com,
mattia.ippoliti@intesasanpaolo.com, christian.mattia@intesasanpaolo.com, davide.corbelletto@intesasanpaolo.com
†Classiq Technologies, Tel Aviv, Israel
orsa@classiq.io, ron@classiq.io, amir@classiq.io
‡DAUIN, Politecnico di Torino, Torino, Italy
emanuele.dri@polito.it, bartolomeo.montrucchio@polito.it

arXiv:2402.05574v3 [quant-ph] 1 Oct 2024

***Abstract*—This work introduces a novel approach to price rainbow options, a type of path-independent multi-asset derivatives, with quantum computers. Leveraging the Iterative Quantum Amplitude Estimation method, we present an end-to-end quantum circuit implementation, emphasizing efficiency by delaying the transition to price space. Moreover, we analyze two different amplitude loading techniques for handling exponential functions. Experiments on the IBM QASM simulator validate our quantum pricing model, contributing to the evolving field of quantum finance.**

***Index Terms*—quantum computing, quantum finance, rainbow options, option pricing**

I. Introduction

Quantum computing entails the promise of a paradigm shift in computational technology, offering the potential for solving certain types of problems more efficiently than classical computers. One sector poised for significant impact is the financial one, where quantum algorithms hold the potential to benefit tasks like risk analysis [1], [2], portfolio optimization [3], and assets pricing [4]–[6].

In finance, a crucial aspect of asset pricing pertains to derivatives. Derivatives are contracts whose value is contingent upon another source, known as the *underlying*. The pricing of options, a specific derivative instrument, involves determining the fair market value (discounted payoff) of contracts affording their holders the right, though not the obligation, to buy (call) or sell (put) one or more underlying assets at a predefined strike price by a specified future expiration date (maturity date). This process relies on mathematical models, considering variables like current asset prices, time to expiration, volatility, and interest rates.

Two strategies are commonly used to identify the price of an option: through mathematical models that are solvable analytically or using Monte Carlo simulations. The former relies on strong assumptions regarding market behavior, while the latter handles more complex scenarios. Monte Carlo methods allow the simulation of the evolution of the assets' price given stochastic parameters and provide an estimate of the fair market value exploiting the central limit theorem [7]. The accuracy of the estimate increases with the number of simulations performed, with the confidence interval scaling as $O(1/\sqrt{M})$, where M represents the number of simulations (samples). The Monte Carlo simulations approach can be computationally intensive for certain derivatives, such as path-dependent options.

Quantum computing, therefore, can be a potential advantage when pricing complex options. Using the Amplitude Estimation algorithm, quadratic fewer samples would be required to reach the same result. Essentially, Amplitude Estimation can estimate a parameter with a convergence rate of $1/M$, where M now is the number of *quantum* samples used. A quantum sample corresponds to an application of the Grover operator, computationally analogous to a classical one. By lowering the complexity of the Grover operator, the theoretical speedup can in principle be efficiently exploited to achieve reduced execution times.

In this regard, the initial proposals refer to [8], in which the authors, for the first time, pioneered a methodology for leveraging the Quantum Amplitude Estimation algorithm in derivative pricing. The article serves as a starting point for subsequent research efforts that have extended and enhanced the proposed approach. Specifically, in [9] the authors developed algorithms for various specific classes of options, including vanilla options, multi-asset options, and path-dependent options. Their emphasis lies on the practical implementation of the necessary operators, accompanied by simulation results and an error mitigation scheme designed for execution on real hardware. Notably, their approach stands out for the utilization of the more efficient Amplitude Estimation without Phase Estimation protocol [10]. More recently, in [11], the authors present an upper bound on the resources necessary to achieve a significant quantum advantage in derivative pricing. Moreover, they address some of the challenges of previous approaches by introducing the re-parameterization method to incorporate stochastic processes.

It is important to note that several recent works focused

on the intricacies of state preparations [12]–[16]. In order to lower the threshold for achieving a quantum advantage, our work focuses instead on the problem of transitioning to the price space (see Section II). This is another module that still presents a high degree of complexity in related works and, to the best of the authors' knowledge, has fewer proposals for optimization in the literature. Furthermore, to allow an estimate of the complexity of an end-to-end approach the authors decided to focus on a type of options for which, to the best of their knowledge, a suitable quantum algorithm has not yet been proposed: rainbow options.

Rainbow options offer the holder the right to buy or sell an underlying asset at a predetermined price by a specified future date, with the unique feature of multiple underlying assets involved in their valuation [17] (each underlying can be thought as a color so their sum results in a "rainbow"). Rainbow options share a common characteristic with correlation options and basket options in that they all involve multiple underlying securities. However, the latter two are tied to a single price determined by those underlying securities. In contrast, rainbow options are designed as calls or puts on the best or worst performer among the underlying assets.

In summary, the main contributions of this work are the following:

- An end-to-end implementation of a quantum circuit for rainbow options pricing using the Iterative Quantum Amplitude Estimation (IQAE) [18] method for estimating the expected payoff, with the advantage of no need for Quantum Phase Estimation [19]. This implementation involved running circuits on a quantum simulator with up to 24 qubits and two correlated assets, using real-world data.
- Additionally, we follow the proposal in [20] and adapt it for rainbow options, delaying the transition from the return space to the price space [11] (see Section II) until the payoff calculation phase, raising the need to encode the result of an exponential function inside the amplitude of an auxiliary qubit (exponential amplitude loading). This way, we save costly arithmetic computation and combine the transition to price space and amplitude loading in a single efficient block. Moreover, we avoid losing precision with digital operations.
- Finally, the paper addresses this crucial aspect of amplitude loading, providing different implementations starting from a generic formulation and narrowing down to the specific case. In this regard, two distinct approaches are proposed: *integration* (for strictly positive or strictly negative monotonous functions), and *direct loading* (for exponential functions).

The advantages and nuances of each approach are presented and discussed in depth.

The rest of this paper is organized as follows: Section II introduces the relevant literature and provides the technical foundation for quantum option pricing. Section III presents our contributions, detailing the methodology behind our algorithm. Section IV contains a detailed analysis of the complexity of the proposed implementation. Then, Section V illustrates the experiments conducted and the relative results. Finally, Section VI contains the final remarks and a summary of our work.

II. Quantum Option Pricing

Quantum computing holds significant promise in achieving algorithmic speedup for various computational tasks. Specifically, Grover's search offers a theoretical quadratic speedup in the search of unstructured databases [19]. Beyond database search, Grover's algorithm has been extended to many diverse applications, among which Amplitude Amplification and Estimation. The former enables to reach a theoretical quadratic speedup for estimating expectation values. This capability holds relevance to problems traditionally addressed using classical Monte Carlo methods [21].

Option pricing primarily revolves around the computation of an expectation value involving a function applied to one or more stochastic financial underlyings. In scenarios extending beyond the Black-Scholes-Merton (BSM) model [22], the prevalent approach for such pricing involves resorting to Monte Carlo evaluation techniques.

In recent years, building on this premise and starting from the proposals in [8], which introduced the earliest quantum algorithm for valuing European and Asian options, several works dealt with the complexities of quantum option pricing [9], [11], [23]. In particular, in [9], the authors describe a structured methodology for pricing several classes of options.

Summarizing the contributions of these relevant studies, we identify three key components for pricing path-independent options with gate-based quantum computers:

1) Encode the probability distribution of a discrete multivariate random variable W taking values in $\{w_0, .., w_{N-1}\}$ describing the assets' prices at the maturity date. The number of discretized values, denoted as N, depends on the precision of the state preparation module and is consequently connected to the number of qubits (n) according to the formula $N = 2^n$. The resulting encoding for the state preparation is the following:
$$\sum_{i=0}^{N-1} \sqrt{p(w_i)}\,|w_i\rangle \tag{1}$$
Notably, in related works [9], [11] the assets' price evolution is assumed to be modeled by a Geometric Brownian Motion. Consequently, underlying asset prices adhere to a log-normal distribution.
On this note, in [11], two strategies for state preparation are discussed. The former involves loading the distribution of asset prices (*price space*), while the latter focuses on loading the distribution of log-returns (*return space*). When asset prices obey a log-normal distribution, then the log-returns are distributed normally. Moving from the former representation to the latter presents a notable advantage, as it involves preparing the state with a normal distribution instead of a log-normal one. This type of state preparation has been extensively explored

in literature, leading to various optimized implementation strategies, as demonstrated in [24]. The authors in [11], performed the state preparation by loading several standard normal distributions (Gaussians) in parallel and subsequently applying affine transformations to obtain the desired multivariate distribution.

2) Construct the circuit which computes a scaled version $\tilde{f}$ of the payoff function f inside the amplitude of a target qubit:

$$\sum_{i=0}^{N-1} \sqrt{(1-\tilde{f}(w_i))p(w_i)}\,|w_i\rangle\,|0\rangle + \sum_{i=0}^{N-1} \sqrt{\tilde{f}(w_i)\,p(w_i)}\,|w_i\rangle\,|1\rangle \tag{2}$$

In [9] the authors encoded directly the payoff inside the amplitude. In contrast, in [11] the function is encoded into a quantum register before being rotated to the amplitude. It is crucial to emphasize that, when the payoff is defined in terms of prices, it is necessary to adopt a strategy for transitioning to the price space before computing the payoff in case of a return space state preparation.

3) Calculate the expectation value of the payoff $\mathbb{E}[f]$ using Amplitude Estimation [25]. The Amplitude Estimation algorithm aims at efficiently estimating a in

$$\mathcal{A}|0\rangle = \sqrt{1-a}\,|\psi_0\rangle\,|0\rangle + \sqrt{a}\,|\psi_1\rangle\,|1\rangle \tag{3}$$

where $\mathcal{A}$ is a unitary operator while $|\psi_0\rangle$ and $|\psi_1\rangle$ are some normalized states. Thus, a is the probability of measuring $|1\rangle$ in the last qubit. Considering $\mathcal{A}$ as the operator that creates the state in Equation 2, the value a to estimate is:

$$a = \sum_{i=0}^{N-1} \tilde{f}(w_i)\,p(w_i) = \mathbb{E}[\tilde{f}] \tag{4}$$

By properly post-processing the estimated a value it is possible to reconstruct the desired expected payoff $\mathbb{E}[f]$ value.

It is worth mentioning that the final step needed for pricing the option is the discounting of the expected value, which is performed classically.

III. Methodology

The building blocks of a simulation for pricing call Rainbow Options through a Monte Carlo approach include: ***(i)*** the evolution of assets' prices, ***(ii)*** selection of the maximum asset price and ***(iii)*** calculation of the payoff, as illustrated in Figure 1.

Under the hypothesis of asset prices being modeled by a Geometric Brownian Motion, at a given t time, the prices follow a multivariate log-normal distribution. Considering $\bar{S}_t$ a vector holding all the asset prices at time t, the distributions of log-returns $\bar{R}_t = \log \bar{S}_t/\bar{S}_0$ (where $\bar{S}_0$ denotes the initial prices of the assets), is normally distributed. It is therefore possible to generate standard Gaussian samples and take advantage of the Cholesky decomposition of the correlation matrix, as outlined in [11], to establish correlations among the samples. After log-returns sampling, as depicted in Figure 1, a transition to the price space becomes necessary to select the asset with the maximum price value ($M_t = \max(\bar{S}_t)$). This involves executing the operation: $\bar{S}_t = \bar{S}_0 e^{\bar{R}_t}$. Our strategy consists of delaying this operation until the payoff calculation, as shown in Figure 2.

Therefore, to identify the asset with the maximum value within the return space, the following statement is used:

$$\arg\max(\bar{S}_0 e^{\bar{R}_t}) = \arg\max(\log(\bar{S}_0) + \bar{R}_t) \tag{5}$$

This holds due to the monotonically increasing nature of the exponential function. Therefore the output of the maximum module, in return space, is determined by the quantity:

$$Z_t = \max(\log(\bar{S}_0) + \bar{R}_t) \tag{6}$$

After the maximum module, in the price space, the payoff is calculated as $v(M_t) = \max(M_t - K, 0)$ or equivalently:

$$v(M_t) = \begin{cases} M_t - K, & \text{if } M_t \geq K \\ 0, & \text{if } M_t < K \end{cases} \tag{7}$$

where K is the strike price.

Since the output of the maximum module in return space is not M_t but the reformulation in Equation 6, the system in return space becomes:

$$v(Z_t) = \begin{cases} e^{Z_t} - K, & \text{if } Z_t \geq \log(K) \\ 0, & \text{if } Z_t < \log(K) \end{cases} \tag{8}$$

It is therefore possible to move the exponential operation to the payoff calculation phase.

The proposed quantum algorithm implements the flow in Figure 2, delaying the exponential module until the payoff calculation phase. In order to minimize resource usage, a rescaling of all the variables with respect to the first asset is performed. To take into account the rescaling, the payoff calculation can be formalized as follows:

$$v(z) = \begin{cases} e^{b\cdot z + b'} - K, & \text{if } z \geq \frac{\log(K)-b'}{b} \\ 0, & \text{if } z < \frac{\log(K)-b'}{b} \end{cases} \tag{9}$$

with $z = \frac{Z_t - b'}{b}$. The value of the constants b and b', will be addressed in Section III-A. By applying the property of linearity of the mean as it follows:

$$\begin{aligned} &\mathbb{E}\left[\max\left(e^{b\cdot z + b'} - K, 0\right)\right] \\ =&\mathbb{E}\left[\max\left(e^{b\cdot z}, Ke^{-b'}\right)\right] e^{b'} - K \end{aligned} \tag{10}$$

we are able to delay some of the operations needed for the case $z \geq \frac{\log(K)-b'}{b}$, in a post-processing phase, executed classically. We propose two different strategies for computing e^{bz} using a quantum circuit having $|z\rangle$ encoded inside a state as a fixed point number.

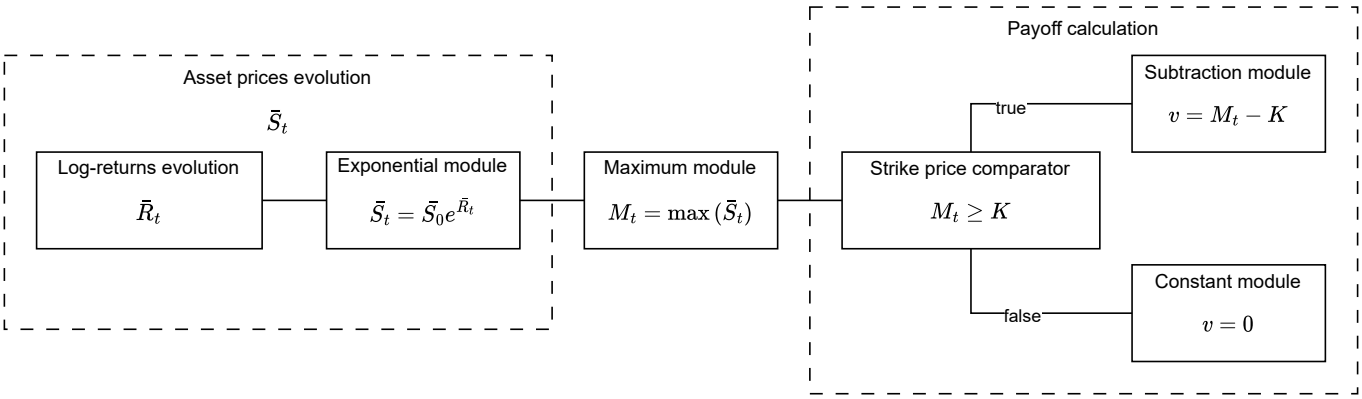

Fig. 1. A single evolution of a path in a Monte Carlo Simulation scenario for pricing rainbow options. The approach shows the *price space* version of the pricing algorithm.

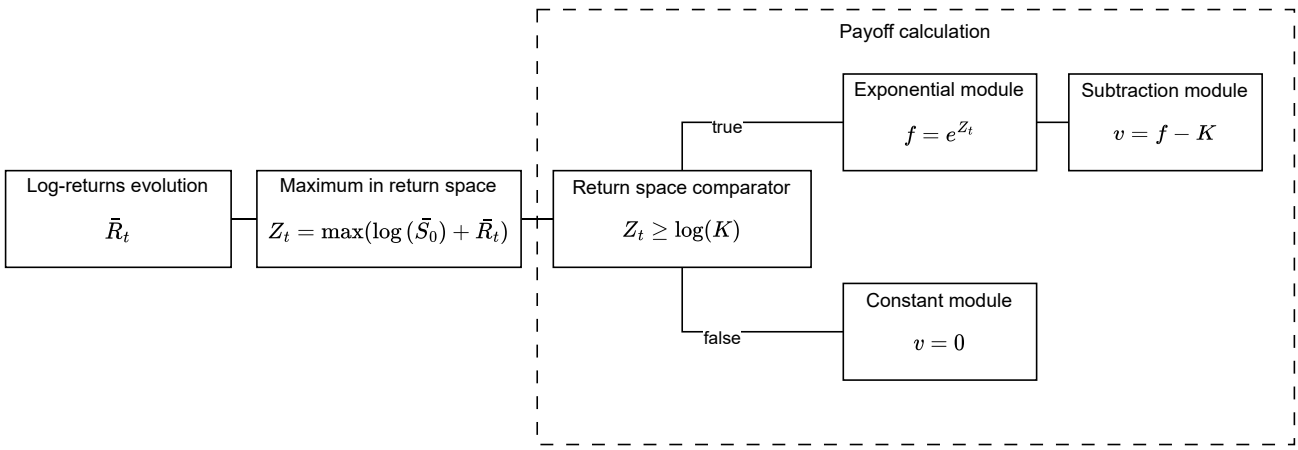

Fig. 2. A single evolution of a path in a Monte Carlo Simulation scenario for pricing rainbow options. The approach shows the *return space* version of the pricing algorithm.

A. Quantum Encoding

The Gaussian state preparation used to load the quantum variables in the return space is encoded as follows:

$$\bigotimes_{i=0}^{N-1} |d_i\rangle \tag{11}$$

where N is the number of considered assets and d_i represents the state of the i-th discrete Gaussian. $|d_i\rangle$ is prepared in the state $\sum_{j=0}^{2^m-1} \sqrt{p(j)}|j\rangle$ where $p(j)$ is the probability of picking the j-th sample from a discrete standard Gaussian and m is the number of qubits dedicated to the discretization. The samples need to be shifted and correlated to obtain the desired multivariate normal distribution. Considering the time delta between the starting date (t_0) and the maturity date (t), we can express the return value R_i for the i-th asset as $R_i = \mu_i + y_i$. Where:

- $\mu_i = (t - t_0)\tilde{\mu}_i$, being $\tilde{\mu}_i$ the expected daily log-return value. It can be estimated by considering the historical time series of log-returns for the i-th asset.
- y_i is obtained through the dot product between the matrix $\mathbf{L}$ and the standard multivariate Gaussian sample:

$$y_i = \Delta x \cdot \sum_k l_{ik} d_k + x_{min} \cdot \sum_k l_{ik} \tag{12}$$

  Δx is the Gaussian discretization step, x_{min} is the lower Gaussian truncation value and $d_k \in [0, 2^m - 1]$ is the sample taken from the k-th standard Gaussian. l_{ik} is the i, k entry of the matrix $\mathbf{L}$, defined as $\mathbf{L} = \mathbf{C}\sqrt{(t - t_0)}$, where $\mathbf{C}$ is the lower triangular matrix obtained by applying the Cholesky decomposition to the historical daily log-returns correlation matrix.

We want to normalize the output value of the maximum module (Z_t from Equation 6) with respect to the first asset.

This results in a maximum module that acts on scaled z_i return space values:

$$z = \max(z_0, ., z_i, ., z_{N-1})$$

where $z_0 = d_0$ and z_i with $i \neq 0$ are:

$$z_i = \frac{R_i + \log(S_0^i) - \mu_0 - \log(S_0^0) - x_{min} l_{0,0}}{\Delta x l_{0,0}} \tag{13}$$

To compute the z_i values and z, the maximum of them, an arithmetic module is used to generate the following state:

$$\sum_{D \in [0, 2^m - 1]^N} \sqrt{p(D)} |D\rangle |z(D)\rangle \tag{14}$$

where D is a N-uple whose i-th element, $d_i \in [0, 2^m - 1]$, represents a sample of the i-th standard Gaussian and $p(D) = \prod_{i=0}^{N-1} p(d_i)$.

Starting from the z value, the inverse operation to apply to obtain Z_t (see Equation 6) are:

$$Z_t = z l_{0,0} \Delta x + \mu_0 + \log(S_0^0) + x_{min} l_{0,0} \tag{15}$$

Recalling Equation 9, b and b' are therefore:

$$b = l_{0,0} \Delta x$$
$$b' = \mu_0 + \log(S_0^0) + x_{min} l_{0,0}$$

We can express z as a function of an integer x: $z = \frac{x}{2^P}$ with P the number of fraction places used to encode z. We can then rewrite Equation 9, considering to perform in the post-processing the operations identified in Equation 10, as:

$$f(x) = \begin{cases} e^{ax}, & \text{if } \frac{x}{2^P} \geq \frac{\log(K) - b'}{b} \\ K e^{-b'}, & \text{if } \frac{x}{2^P} < \frac{\log(K) - b'}{b} \end{cases} \tag{16}$$

where $a = \frac{b}{2^P}$.

We consider to load a function $\tilde{f}$, that is a modified version of f, inside the amplitude using an auxiliary qubit:

$$\begin{aligned} &\sum_{i=0}^{2^R - 1} \sqrt{p(x_i)} \sqrt{\tilde{f}(x_i)} |x_i\rangle |\psi_1\rangle |1\rangle + \\ &\sum_{i=0}^{2^R - 1} \sqrt{p(x_i)} \sqrt{1 - \tilde{f}(x_i)} |x_i\rangle |\psi_0\rangle |0\rangle \end{aligned} \tag{17}$$

where $|\psi_1\rangle$ and $|\psi_0\rangle$ represent additional qubits inside the circuit that have no impact on the Amplitude Estimation outcome, such as the qubits exploited for loading the normal distribution $|D\rangle$. R is the size of $|x\rangle$ register. Amplitude Estimation is used to compute the expected payoff value, collecting the probability of having a $|1\rangle$ in the auxiliary. The post-processing is then needed to obtain $\mathbb{E}[f]$ from $\mathbb{E}[\tilde{f}]$ and finally $\mathbb{E}[v]$ through the one defined in Equation 10.

To implement $\tilde{f}$, a comparator is used to evaluate the condition $\frac{x}{2^P} \geq \frac{\log(K) - b'}{b}$ and the outcome is exploited as a control for the two cases modules. If the condition is not verified, a rotation is applied to the target qubit to ensure that the amplitude of $|1\rangle$ contains the desired constant value $\tilde{H}$, which is related to $H = K e^{-b'}$. The needed rotation angle is therefore $\theta = 2 \arcsin\left(\sqrt{\tilde{H}}\right)$. If otherwise the condition is satisfied, the block performing the exponential amplitude loading (the loading of $\sqrt{\tilde{h}}$, with $\tilde{h}$ a modified version of $h(x) = e^{ax}$, inside the $|1\rangle$ amplitude) is activated.

We faced two approaches for exponential amplitude loading: direct exponential amplitude loading and integration amplitude loading. Both of them require an auxiliary register $|r\rangle$ of the same size (R) of $|z\rangle$.

B. Direct Exponential Amplitude Loading

The direct exponential amplitude loading enables encoding the following function in $\tilde{f}$:

$$\tilde{f}(x) = \begin{cases} e^{-a\hat{x}}, & \text{if } \frac{x}{2^P} \geq \frac{\log(K) - b'}{b} \\ K e^{-(b' + a x_{max})}, & \text{if } \frac{x}{2^P} < \frac{\log(K) - b'}{b} \end{cases} \tag{18}$$

where $\hat{x}$ is the binary complement of x ($\hat{x} = x_{max} - x$) and $x_{max} = 2^R - 1$, the maximum value that can be stored in $|x\rangle$ register. We need to add to the post-processing operation a term:

$$\begin{aligned} &\mathbb{E}\left[\max\left(e^{b \cdot z}, K e^{-b'}\right)\right] e^{b'} - K \\ =&\mathbb{E}\left[\max\left(e^{-a\hat{x}}, K e^{-b' - a x_{max}}\right)\right] e^{b' + a x_{max}} - K \end{aligned} \tag{19}$$

For loading $e^{-a\hat{x}}$, $|r\rangle$ register is initialized to $|0\rangle$ and one controlled rotation for each qubit is performed as shown in Figure 3. The rotations angles are: $\theta_i = 2 \arccos\left(\sqrt{e^{-a 2^i}}\right)$. All the probabilities of getting a $|0\rangle$ in the $|r\rangle$ register are then collected by a multi-controlled X (MCX) gate and stored in the $|1\rangle$ state of a target qubit.

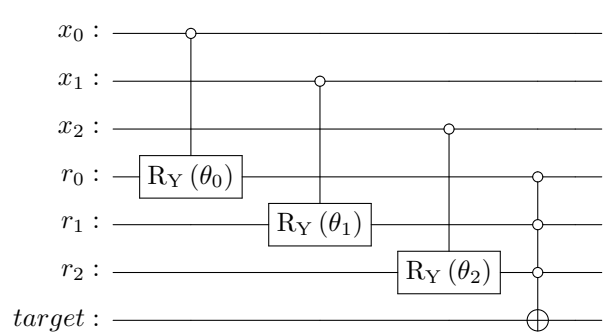

Fig. 3. Circuit implementing the direct exponential amplitude loading. The $|x\rangle$ register controls rotations on the auxiliary $|r\rangle$ register, initialized to $|0\rangle$, to obtain the loading of $e^{-a\hat{x}}$, with $\hat{x}$ binary complement of x, inside the $|0\rangle$ state. An MCX is used to transfer the probability of getting a $|0\rangle$ inside the $|r\rangle$ register to the amplitude of the target qubit, conditioned on the corresponding $|x\rangle$ state probabilities.

C. Integration Amplitude Loading

The second method, the integration amplitude loading, is inspired by the work in [26], the aim is to use the outcome of a comparator as an integrator to generate $\tilde{h}$ ($\tilde{h}$ is a modified

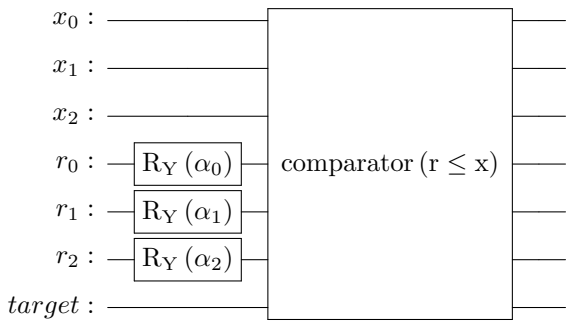

Fig. 4. Circuit implementing the integration amplitude loading. The exponential state preparation acts with R_y gates on the $|r\rangle$ register initialized to $|0\rangle$. Then a comparator is used to collect the probability for the $|r\rangle$ register to be lower than the $|x\rangle$ one. In such a way an integration is performed over the state-prepared amplitudes and weighted for the related $|x\rangle$ probabilities.

version of h). The comparator collects the probabilities $g(r)$ of $|r\rangle$ state until $|r\rangle$ register is lower than $|x\rangle$:

$$\begin{aligned}&\sum_{r=0}^{2^R-1}\sqrt{g(r)}|x\rangle|r\rangle|r\leq x\rangle\\&=|x\rangle\otimes\left[\sum_{r=0}^{x}\sqrt{g(r)}|r\rangle|1\rangle+\sum_{r=x}^{2^R-1}\sqrt{g(r)}|r\rangle|0\rangle\right]\end{aligned}\tag{20}$$

Collecting the probability to have $r \leq x$ we can define the function:

$$\tilde{h}(x)=\sum_{r=0}^{x}g(r)\tag{21}$$

Evaluating the probability to get a $|1\rangle$ results in $\sum_{x=0}^{2^R-1}\tilde{h}(x)$. To obtain a given function $\tilde{h}$ a proper function $g(r)$ should be chosen. The $g(r)$ for $r=0$ value must therefore be $g(0)=\tilde{h}(0)$ and for all the other r:

$$g(r)=\tilde{h}(r)-\tilde{h}(r-1)$$

We first hypothesized to train a variational circuit to perform the state preparation for $g(r)$. This approach can be used for every monotonous function h that is strictly positive or strictly negative. However, there is a faster method to load the exponential function $h(x)=e^{ax}$ if we consider

$$\tilde{h}(x)=\frac{h(x+1)-h(0)}{h(x_{max}+1)-h(0)}=\frac{e^{a(x+1)}-1}{e^{a(x_{max}+1)}-1}\tag{22}$$

getting

$$g(r)=\frac{e^{ar}}{\sum_{j=0}^{x_{max}}e^{aj}}\tag{23}$$

these values can be loaded using a circuit performing an exponential state preparation [27] of unitary depth and no controlled operations. The circuit is shown in Figure 4 where $\alpha_i=2\arctan\left(e^{\frac{a2^i}{2}}\right)$. If we therefore consider:

$$\tilde{f}(x)=\begin{cases}\frac{e^{a(x+1)}-1}{e^{a(x_{max}+1)}-1}, & \text{if } \frac{x}{2^P}\geq\frac{\log(K)-b'}{b}\\ \frac{Ke^{-b'}}{c}-\frac{e^{-a}}{c}, & \text{if } \frac{x}{2^P}<\frac{\log(K)-b'}{b}\end{cases}\tag{24}$$

with $c=\frac{e^{a(x_{max}+1)}-1}{e^a}$, the relative updates to the post-processing parameters follow:

$$\begin{aligned}&\mathbb{E}\left[\max\left(\frac{e^{a(x+1)}-1}{e^{a(x_{max}+1)}-1}c+\frac{1}{e^a},Ke^{-b'}\right)\right]e^{b'}-K\\&=\mathbb{E}\left[\max\left(\frac{e^{a(x+1)}-1}{e^{a(x_{max}+1)}-1},\frac{Ke^{-b'}}{c}-\frac{e^{-a}}{c}\right)\right]ce^{b'}\\&\qquad+e^{b'}e^{-a}-K\end{aligned}\tag{25}$$

Algorithm 1 Algorithm

Input: Mean of daily log-return of the n assets,
Desired Strike Price K,
Time to expiration dt,
Lower triangular matrix from the Cholesky decomposition of log-returns covariance matrix

for each n assets **do**
  Load Gaussian Distribution ($\mu=1,\sigma=0$)
end for
Apply the affine transformation and compute the maximum between all samples
Apply the comparator operator between $\frac{(log(K)-b')}{b}$ and the maximum output
if Integration Method **then**
  Load the Exponential State Preparation
  Apply the comparator between the maximum and the exponential state controlled by the first comparator $|1\rangle$ state and store the comparison result inside the target qubit
end if
if Direct Method **then**
  Apply R_y gates controlled by each qubit of the maximum $|0\rangle$ state
  Apply a MCX gate controlled by the loaded distribution $|0\rangle$ state and the first comparator $|1\rangle$ state to the target qubit
end if
Apply the R_y gate controlled by the first comparator $|0\rangle$ state to the target qubit
Use Quantum Amplitude Estimation to extract the probability of the target qubit to be $|1\rangle$

Output: Expected Payoff for the option by applying the post-processing to the QAE result

IV. Complexity Analysis

Since fault tolerant implementations of the T gate are an expensive resource, minimizing the number of T stages, referred to as the T-depth, is an important target in algorithm development [28]. Our payoff implementation's T-depth is smaller compared to other proposed implementations of the same functionality [11]. In this section, we analyze the T-depth of the payoff function for both the direct and integration methods. We do not analyze the assets state preparation

module and the arithmetic computing their maximum since they are not the focus of this article.

Regarding machine precision, we assume the same register size k for each quantum register in the different functions, as they all have the same domain and units.

The first function that is common to both methods is the $z \geq \frac{\log(K)-b'}{b}$ comparator. It compares the z output of the arithmetic maximum module, with a predefined constant $\frac{\log(K)-b'}{b}$. The Toffoli-depth of a comparator between a quantum register and another quantum register or a constant [29] is $2\log_2(k)+5$. We analyze the Toffoli gate depth, assuming an unlimited number of qubits, and multiply it by a factor of 3 T-depth per Toffoli gate [30].

$$D^T_{comp}(k) = D^{Toffoli}_{comp}(k) * 3 = 6\log_2(k) + 15 \tag{26}$$

For both methods, there is a controlled rotation if the condition is not verified. A controlled R_y gate is composed of two CX gates and two R_y gates. According to Solovay–Kitaev theorem, each R_y gate has a T-depth of:

$$D^T_{CR_y}(\epsilon_{R_y}) = 3\log_2(1/\epsilon_{R_y}) \tag{27}$$

where ϵ_{R_y} is the infidelity of the R_y gate [31]. In the next sections, a detailed analysis of the two methods will be reported.

A. Direct Exponential Method

For the direct exponential method, after the comparator and the controlled rotation, a direct exponential amplitude loading is implemented with k controlled R_y gates in parallel. Demanding payoff infidelity of ϵ_{payoff} requires stricter ϵ_{R_y} infidelity for each of the controlled R_y gates. The payoff infidelity from loading using k controlled R_y gates is $2k\epsilon_{R_y}$, and if considering the the previous R_y gate, the infidelity is $\epsilon_{payoff} = 2(k+1)\epsilon_{R_y}$. Therefore, after considering the demand for infidelity, the depth of the exponential loading part and the controlled R_y part is:

$$\begin{aligned} &D^T_{exp}(\epsilon_{payoff}) + D^T_{CR_y}(\epsilon_{payoff}) \\ &= 2 \cdot D^T_{CR_y}(\epsilon_{payoff}) = 12\log_2\left(\frac{2(k+1)}{\epsilon_{payoff}}\right) \end{aligned} \tag{28}$$

The loading block is followed by a multi-controlled X gate, with $k+1$ control qubits. A reduced T-depth of the MCX is achieved using relative Toffoli gates [32]. The Toffoli gate depth optimization is obtained through the use of the Classiq platform [33], resulting in a T-depth of $14\log_3(n/2)+5$ where n is the number of controls, equal to $k+1$ in our case.

$$D^T_{MCX}(k) = 14\log_3\left(\frac{k+1}{2}\right) + 5 \tag{29}$$

The final T-depth of the payoff function for the exponential method is:

$$\begin{aligned} &D^T_{exponential} = D^T_{comp} + D^T_{exp} + D^T_{MCX} + D^T_{CR_y} \\ &= 6\log_2(k) + 14\log_3\left(\frac{k+1}{2}\right) + 12\log_2\left(\frac{2(k+1)}{\epsilon_{payoff}}\right) + 20 \end{aligned} \tag{30}$$

B. Integration Method

For the integration method, after the comparator and the controlled rotation, we use an integrator, which is a controlled register-to-register comparator. We do not consider the depth of the exponential state preparation before this integrator, since it is parallel to the assets state preparation block and the maximum arithmetic block. This comparator (the integrator) is controlled and therefore implemented by an uncontrolled subtractor, its uncomputation, and a controlled sign check. The sign check, which is a CX gate, becomes a CCX gate in its controlled version. Therefore the T-depth becomes:

$$D^T_{integ}(k) = \left(D^{Toffoli}_{comp}(k) + 1\right) * 3 = 6\log_2(k) + 18 \tag{31}$$

To conclude, when using the integration method, we assume a summation of T-depth from each function:

$$\begin{aligned} D^T_{integration} &= D^T_{comp} + D^T_{integ} + D^T_{CR_y} \\ &= 12\log_2(k) + 6\log_2\left(\frac{2}{\epsilon_{payoff}}\right) + 33 \end{aligned} \tag{32}$$

C. Infidelity and Resolution Demand

The total estimation infidelity ($\tilde{\epsilon}$) in the probability domain, before post-processing, is a summation of:

- ϵ_{QAE}, the error related to the Amplitude Estimation algorithm
- ϵ_{payoff}, the payoff function infidelity
- ϵ_{GSP}, the infidelity of the Gaussian state preparation
- ϵ_{max}, the infidelity of the arithmetic computing the affine transformation and the maximum

The post-processing scaling factor linearly amplifies this infidelity, and it is essential to consider this effect as well. Considering our post-processing strategy, the delay in subtracting the strike price, as opposed to approaches where it is factored into the payoff calculation, results in a different impact on infidelity. In particular, if a precision ϵ is required on the estimated value, the total infidelity $\tilde{\epsilon}$ should be lower than a quantity that depends on the post-processing. With no delayed subtraction, the infidelity of the estimate is affected as:

$$\tilde{\epsilon} \leq \frac{\epsilon}{S_{max} - K} \tag{33}$$

Where S_{max} is the maximum of the discretized values assets can take. The infidelity demand for the integration method is:

$$\tilde{\epsilon} \leq \frac{\epsilon}{S_{max} - e^{b'-a}} \tag{34}$$

For the direct method, the infidelity is:

$$\tilde{\epsilon} \leq \frac{\epsilon}{S_{max}} \tag{35}$$

The delayed strike price subtraction has therefore a different impact for the two methods that depends on the input parameters of the pricing problem.

V. Experiments

The algorithm was implemented using The Classiq Platform [27]. The platform allowed abstraction by functionally describing the algorithm using high-level building blocks. Each of the parts of the algorithm (state preparation, payoff function, and Amplitude Estimation) was designed through a high-level model. Signed fixed-point quantum variables were used, including the appropriate arithmetic operations over them.

The platform optimizes the quantum program functions by storing multiple implementations for the same functionality and automatically choosing when to uncompute expressions to free auxiliary qubits and re-use them. This compilation approach allowed the scaling of the problem size (number of assets, resolution of distribution and Amplitude Estimation, payoff function complexity) in a given quantum simulation width limit.

TABLE I
Assets' initial price for the given example

Asset	S_0
1	\$ 193.97
2	\$ 189.12

TABLE II
Problem parameters for the given example

Asset	μ	Cov
1	$5.096 \cdot 10^{-4}$	$[3.35, 2.57] \cdot 10^{-4}$
2	$6.255 \cdot 10^{-4}$	$[2.57, 4.18] \cdot 10^{-4}$

The experiments were conducted using the IBM *QASM* noiseless simulator. For each experiment, we ran 1000 shots and measured the objective qubit after the IQAE subroutine [18]. As a proof of concept, we tested our algorithm with two assets whose value at the initial time $t = 0$ can be seen in Table I and a strike price $K = \$190$. The relative log-returns mean and covariance matrix are visible in Table II. Finally, a time to expiration of 250 days (maturity date - starting date) was supposed for the option.

The code used for running these experiments is available in [34].

Two qubits were used for each Gaussian distribution. The arithmetic was rounded to one decimal place. An additional qubit was required for the comparator, and the exponential register was sized to match the output of the arithmetic. Finally, one target qubit was used to encode the payoff for the Amplitude Estimation. The Classiq platform also assigned the necessary ancilla qubits, resulting in a total of 24 qubits.

To validate the model, we compared the results obtained using the quantum simulator with the classically computed expected value. The classical estimation process considers both the discretization of the Gaussian distributions and the arithmetic precision, in order to achieve a fair comparison. When using as input the data in Table I, the corresponding expected payoff is $\$23.0238$.

The simulations were performed using IQAE with a precision $\epsilon = 0.01$ and (for the confidence interval) $\alpha = 0.05$. The results are obtained following the procedure shown in Algorithm 1 and can be seen in Figure 5. As can be observed, the expected result was in both cases well within the confidence interval of the simulation.

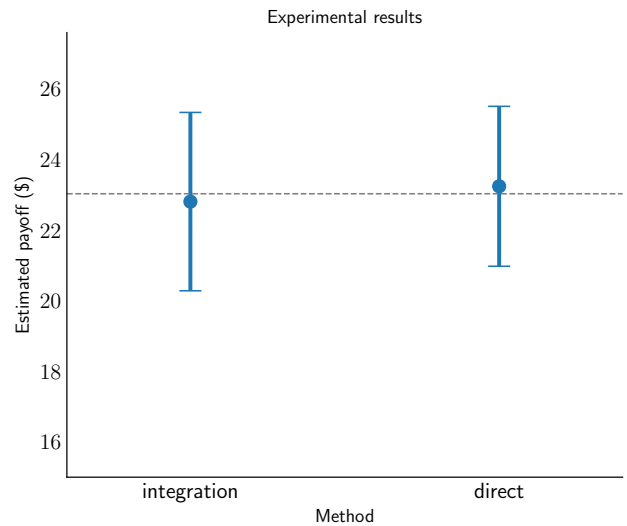

Fig. 5. Estimated payoff (with relative confidence interval) using the two proposed methodologies for amplitude loading and the IBM QASM simulator. The dotted line represents the classically computed expected value.

VI. Conclusion

In this paper, we explored the application of quantum computing to the pricing of rainbow options, a derivative product with multiple underlying assets. The traditional methods for pricing options involve computationally intensive Monte Carlo simulations, making them a potential candidate for quantum acceleration through the use of Amplitude Estimation.

Our work builds upon previous research in quantum derivative pricing. We presented an end-to-end implementation for our proposed quantum algorithm, addressing the complexity associated with transitioning to the price space. The key contributions of our work include:

- Quantum Circuit Implementation: development and implementation of a quantum circuit for rainbow option pricing using the Iterative Quantum Amplitude Estimation (IQAE) method.
- Efficient Transition Handling: implementation of a tailor-made strategy (focusing on rainbow options) for delaying the transition from return space to price space until the payoff calculation phase, improving computational efficiency and extending the recent proposal from [20].
- Exponential Amplitude Loading: introduction and exploration of two distinct approaches for handling exponential functions in quantum circuits. Both the approaches, namely the integration amplitude loading and the direct amplitude loading, require loading the desired function inside a qubit's amplitude. Direct exponential loading is suitable only for exponential functions, while integration amplitude loading can handle strictly positive or strictly

negative monotonous functions. The versatility of the latter extends its potential applications to other algorithms, including HHL [35]).

- Validation Experiments: execution of experiments on a two-asset scenario using the IBM QASM simulator to validate the quantum pricing model by comparing its results with those computed classically.

It's important to note that our contributions hold significance for various types of options. As a result, potential future directions could involve addressing more intricate option classes, such as path-dependent options, by leveraging the modules and approaches outlined in this study. Additionally, enhancing the scalability of the existing implementation and conducting small-scale tests on accessible noisy quantum hardware could be considered. Finally, the authors foresee extending the analysis of resource requirements to the entire circuit in the future, aiming to provide more practical assessments of end-to-end implementation needs.

Acknowledgment

The authors would like to thank G. Bella for sharing knowledge on the Monte Carlo algorithm for pricing rainbow options.

The authors would also like to thank E. Cornfeld for the fruitful discussion about optimizing the T-depth of multi-controlled X gates and G. Kishony for the prolific discussion about the T-depth counting of various quantum gates.

References

[1] D. J. Egger, R. G. Gutierrez, J. Mestre, and S. Woerner, "Credit risk analysis using quantum computers," IEEE Transactions on Computers, vol. 70, no. 12, pp. 2136–2145, dec 2021.

[2] E. Dri, A. Aita, E. Giusto, D. Ricossa, D. Corbelletto, B. Montrucchio, and R. Ugoccioni, "A more general quantum credit risk analysis framework," Entropy, vol. 25, no. 4, p. 593, Mar. 2023. [Online]. Available: http://dx.doi.org/10.3390/e25040593

[3] P. K. Barkoutsos, G. Nannicini, A. Robert, I. Tavernelli, and S. Woerner, "Improving variational quantum optimization using cvar," Quantum, vol. 4, p. 256, Apr. 2020. [Online]. Available: http://dx.doi.org/10.22331/q-2020-04-20-256

[4] R. Orús, S. Mugel, and E. Lizaso, "Quantum computing for finance: Overview and prospects," Reviews in Physics, vol. 4, p. 100028, Nov. 2019. [Online]. Available: https://doi.org/10.1016/j.revip.2019.100028

[5] D. J. Egger, C. Gambella, J. Marecek, S. McFaddin, M. Mevissen, R. Raymond, A. Simonetto, S. Woerner, and E. Yndurain, "Quantum computing for finance: State-of-the-art and future prospects," IEEE Transactions on Quantum Engineering, vol. 1, pp. 1–24, 2020.

[6] D. Herman, C. Googin, X. Liu, Y. Sun, A. Galda, I. Safro, M. Pistoia, and Y. Alexeev, "Quantum computing for finance," Nature Reviews Physics, vol. 5, no. 8, pp. 450–465, Jul. 2023. [Online]. Available: https://doi.org/10.1038/s42254-023-00603-1

[7] P. P. Boyle, "Options: A Monte Carlo approach," Journal of Financial Economics, vol. 4, no. 3, pp. 323–338, May 1977. [Online]. Available: https://ideas.repec.org/a/eee/jfinec/v4y1977i3p323-338.html

[8] P. Rebentrost, B. Gupt, and T. R. Bromley, "Quantum computational finance: Monte carlo pricing of financial derivatives," Physical Review A, vol. 98, no. 2, Aug. 2018. [Online]. Available: http://dx.doi.org/10.1103/PhysRevA.98.022321

[9] N. Stamatopoulos, D. J. Egger, Y. Sun, C. Zoufal, R. Iten, N. Shen, and S. Woerner, "Option pricing using quantum computers," Quantum, vol. 4, p. 291, Jul. 2020. [Online]. Available: http://dx.doi.org/10.22331/q-2020-07-06-291

[10] Y. Suzuki, S. Uno, R. Raymond, T. Tanaka, T. Onodera, and N. Yamamoto, "Amplitude estimation without phase estimation," Quantum Information Processing, vol. 19, no. 2, Jan. 2020. [Online]. Available: http://dx.doi.org/10.1007/s11128-019-2565-2

[11] S. Chakrabarti, R. Krishnakumar, G. Mazzola, N. Stamatopoulos, S. Woerner, and W. J. Zeng, "A threshold for quantum advantage in derivative pricing," Quantum, vol. 5, p. 463, Jun. 2021. [Online]. Available: http://dx.doi.org/10.22331/q-2021-06-01-463

[12] A. G. Rattew and B. Koczor, "Preparing arbitrary continuous functions in quantum registers with logarithmic complexity," 2022. [Online]. Available: https://arxiv.org/abs/2205.00519

[13] S. McArdle, A. Gilyén, and M. Berta, "Quantum state preparation without coherent arithmetic," 2022. [Online]. Available: https://arxiv.org/abs/2210.14892

[14] G. Marin-Sanchez, J. Gonzalez-Conde, and M. Sanz, "Quantum algorithms for approximate function loading," Physical Review Research, vol. 5, no. 3, Aug. 2023. [Online]. Available: http://dx.doi.org/10.1103/PhysRevResearch.5.033114

[15] A. G. Rattew and P. Rebentrost, "Non-linear transformations of quantum amplitudes: Exponential improvement, generalization, and applications," 2023. [Online]. Available: https://arxiv.org/abs/2309.09839

[16] G. Marin-Sanchez, J. Gonzalez-Conde, and M. Sanz, "Quantum algorithms for approximate function loading," Phys. Rev. Res., vol. 5, p. 033114, Aug 2023. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevResearch.5.033114

[17] B. A. Eales, Regular and Exotic Options. Macmillan Education UK, 2000, p. 140–177. [Online]. Available: http://dx.doi.org/10.1007/978-1-349-27856-5_6

[18] D. Grinko, J. Gacon, C. Zoufal, and S. Woerner, "Iterative quantum amplitude estimation," npj Quantum Information, vol. 7, no. 1, Mar. 2021. [Online]. Available: http://dx.doi.org/10.1038/s41534-021-00379-1

[19] M. A. Nielsen and I. L. Chuang, Quantum Computation and Quantum Information: 10th Anniversary Edition, 10th ed. USA: Cambridge University Press, 2011.

[20] N. Stamatopoulos and W. J. Zeng, "Derivative pricing using quantum signal processing," 2023.

[21] A. Montanaro, "Quantum speedup of monte carlo methods," Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences, vol. 471, no. 2181, p. 20150301, Sep. 2015. [Online]. Available: http://dx.doi.org/10.1098/rspa.2015.0301

[22] F. Black and M. S. Scholes, "The pricing of options and corporate liabilities," Journal of Political Economy, vol. 81, pp. 637 – 654, 1973. [Online]. Available: https://api.semanticscholar.org/CorpusID:154552078

[23] A. Martin, B. Candelas, A. Rodríguez-Rozas, J. D. Martín-Guerrero, X. Chen, L. Lamata, R. Orús, E. Solano, and M. Sanz, "Toward pricing financial derivatives with an ibm quantum computer," Phys. Rev. Res., vol. 3, p. 013167, Feb 2021. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevResearch.3.013167

[24] A. G. Rattew, Y. Sun, P. Minssen, and M. Pistoia, "The efficient preparation of normal distributions in quantum registers," Quantum, vol. 5, p. 609, Dec. 2021. [Online]. Available: http://dx.doi.org/10.22331/q-2021-12-23-609

[25] G. Brassard, P. Høyer, M. Mosca, and A. Tapp, "Quantum amplitude amplification and estimation," Quantum Computation and Information, p. 53–74, 2002. [Online]. Available: http://dx.doi.org/10.1090/conm/305/05215

[26] R. Krishnakumar and W. Zeng, "Quantum rounding," 2021.

[27] Classiq Technologies Ltd, "Classiq platform." [Online]. Available: https://platform.classiq.io

[28] P. Niemann, A. Gupta, and R. Drechsler, "T-depth optimization for fault-tolerant quantum circuits," in 2019 IEEE 49th International Symposium on Multiple-Valued Logic (ISMVL). IEEE, 2019, pp. 108–113.

[29] T. G. Draper, S. A. Kutin, E. M. Rains, and K. M. Svore, "A logarithmic-depth quantum carry-lookahead adder," arXiv preprint quant-ph/0406142, 2004.

[30] J. Lee, S. Lee, Y.-S. Lee, and D. Choi, "T-depth reduction method for efficient sha-256 quantum circuit construction," IET Information Security, vol. 17, no. 1, pp. 46–65, 2023.

[31] N. J. Ross and P. Selinger, "Optimal ancilla-free clifford+ t approximation of z-rotations," arXiv preprint arXiv:1403.2975, 2014.

[32] D. Maslov, "Advantages of using relative-phase toffoli gates with an application to multiple control toffoli optimization," Physical Review A, vol. 93, no. 2, p. 022311, 2016.
[33] "Multi-control-x Classiq," https://docs.classiq.io/latest/user-guide/function-library/builtin-functions/mcx/, accessed: 2024-01-02.
[34] F. Cibrario, O. S. Golan, G. Ranieri, E. Dri, M. Ippoliti, and R. Cohen, "Quantum rainbow option pricing code implementation," Mar. 2024. [Online]. Available: https://dub.sh/code-repository
[35] A. W. Harrow, A. Hassidim, and S. Lloyd, "Quantum algorithm for linear systems of equations," Phys. Rev. Lett., vol. 103, p. 150502, Oct 2009. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevLett.103.150502